XCPC Team Notebook · 个人板子手册

维护者: mirai_39 / mirai0914 | CCPC / ICPC 冲银 | 纯血 vector · struct 自闭环封装 · 竞赛风

【本地防爆栈编译】：`g++ -O2 -std=c++20 solve.cpp -Wl,--stack=268435456 -o solve.exe`
【快速 I/O】：`cin.tie(nullptr)->sync_with_stdio(false);`

1. 欧拉线性筛 (基础素数筛 + 扩展积性函数 phi & mu)

- φ(p) = p-1; i%p==0 ⇒ φ(ip) = φ(i)*p; 否则 φ(i)*(p-1)
- μ(1)=1, μ(p)=-1; i%p==0 ⇒ μ(ip) = 0; 否则 -μ(i)

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

/**
 * ==========================================================================
 * 模块 1: 基础欧拉筛 (素数线性筛)
 * ==========================================================================
 */
struct EulerSieve {
    int n = 0;
    vector<int> prm;
    vector<bool> isp; // isp[x] == false 表示 x 是质数 (与主人原版逻辑一致)

    EulerSieve(int max_n = 0) {
        if (max_n > 0) build(max_n);
    }

    void build(int max_n) {
        n = max_n;
        prm.clear();
        isp.assign(n + 1, false);

        if (n >= 0) isp[0] = true;
        if (n >= 1) isp[1] = true;

        for (int i = 2; i <= n; i++) {
            if (!isp[i]) {
                prm.push_back(i);
            }
            for (int p : prm) {
                if (1LL * i * p > n) break;
                isp[i * p] = true;
                if (i % p == 0) break;
            }
        }
    }

    // 判定 x 是否为质数 (O(1))
    bool is_prime(int x) const {
        if (x < 2 || x > n) return false;
        return !isp[x];
    }
};

/**
 * ==========================================================================
 * 模块 2: 扩展欧拉筛 (素数 + 欧拉函数 phi + 莫比乌斯函数 mu)
 * ==========================================================================
 * 转移推导备忘 (p 为质数):
 * 1. 欧拉函数 phi:
 *    - phi[p] = p - 1
 *    - 若 i % p == 0: phi[i * p] = phi[i] * p
 *    - 若 i % p != 0: phi[i * p] = phi[i] * (p - 1)
 *
 * 2. 莫比乌斯函数 mu:
 *    - mu[1] = 1, mu[p] = -1
 *    - 若 i % p == 0: mu[i * p] = 0 (含有平方因子 p^2)
```

```
 *    - 若 i % p != 0: mu[i * p] = -mu[i]
 */
struct MultiplicativeSieve {
    int n = 0;
    vector<int> prm;
    vector<bool> isp;
    vector<int> phi;
    vector<int> mu;

    MultiplicativeSieve(int max_n = 0) {
        if (max_n > 0) build(max_n);
    }

    void build(int max_n) {
        n = max_n;
        prm.clear();
        isp.assign(n + 1, false);
        phi.assign(n + 1, 0);
        mu.assign(n + 1, 0);

        if (n >= 0) isp[0] = true;
        if (n >= 1) {
            isp[1] = true;
            phi[1] = 1;
            mu[1] = 1;
        }

        for (int i = 2; i <= n; i++) {
            if (!isp[i]) {
                prm.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : prm) {
                if (1LL * i * p > n) break;
                isp[i * p] = true;
                if (i % p == 0) {
                    // p 是 i 的最小质因数，i 已经包含质因子 p
                    phi[i * p] = phi[i] * p;
                    mu[i * p] = 0;
                    break;
                } else {
                    // p 与 i 互质，由积性函数性质转移
                    phi[i * p] = phi[i] * (p - 1);
                    mu[i * p] = -mu[i];
                }
            }
        }
    }
};
```

2. 快速数论变换 (NTT 多项式乘法 / 快速幂 / 分治连乘)

- MOD = 998244353, 原根 G = 3
- mult: O((N+M)log(N+M)); poly_pow: O(L log L log k); poly_prod: O(N log^2 N)

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

/**
 * @brief 快速数论变换 (NTT)
 * @details 模数固定 998244353，原根 G = 3
 */
struct NTT
{
    static constexpr int MOD = 998244353;
    const int G = 3;

    // 功能：快速幂 a^b % MOD
    // 传参：a 底数，b 指数
    long long ksm(long long a, long long b) const {
        long long res = 1;
        a %= MOD;
```

```
    for (; b; b >>= 1, a = a * a % MOD) {
        if (b & 1) res = res * a % MOD;
    }
    return res;
}

// 功能: 费马小定理求逆元
// 传参: a 待求逆元的整数
long long inv(long long a) const {
    return ksm(a, MOD - 2);
}

// 功能: NTT / INTT 核心变换 (系数 <-> 点值)
// 传参: a 系数数组 (长度需为 2^k), inv_flag 0: 正变换 / 1: 逆变换
void transf(vector<int>& a, bool inv_flag) const {
    int n = a.size();
    if (n <= 1) return;

    vector<int> rev(n, 0);
    int bit = __builtin_ctz(n);

    for (int i = 0; i < n; ++i) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (bit - 1));
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        long long wlen = ksm(G, (MOD - 1) / len);
        if (inv_flag) wlen = inv(wlen);
        int half = len >> 1;

        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < half; ++j) {
                int u = a[i + j];
                int v = 1LL * a[i + j + half] * w % MOD;
                a[i + j] = (u + v >= MOD ? u + v - MOD : u + v);
                a[i + j + half] = (u - v < 0 ? u - v + MOD : u - v);
                w = w * wlen % MOD;
            }
        }
    }

    if (inv_flag) {
        long long inv_n = inv(n);
        for (int i = 0; i < n; ++i) {
            a[i] = 1LL * a[i] * inv_n % MOD;
        }
    }
}

// 功能: 多项式乘法 (卷积) C(x) = A(x) * B(x)
// 传参: a, b 多项式系数向量 (下标 i 为 x^i 项系数)
// 返回: 卷积后的系数向量 (size = deg(A) + deg(B) + 1)
vector<int> mult(vector<int> a, vector<int> b) const {
    if (a.empty() || b.empty()) return {};
    int sz = a.size() + b.size() - 1;
    int n = 1;
    while (n < sz) n <<= 1;

    a.resize(n, 0);
    b.resize(n, 0);

    transf(a, 0);
    transf(b, 0);
    for (int i = 0; i < n; ++i) {
        a[i] = 1LL * a[i] * b[i] % MOD;
    }
    transf(a, 1);

    a.resize(sz);
    return a;
}

// 功能: 多项式快速幂 A^k(x)
```

```
    // 传参: a 底数多项式系数, k 幂次, lim 最高次数限制 (截断到 <= lim 项, 默认 -1 不截断)
    // 返回: 结果多项式系数向量
    vector<int> poly_pow(vector<int> a, long long k, int lim = -1) const {
        vector<int> res = {1};
        while (k > 0) {
            if (k & 1) {
                res = mult(res, a);
                if (lim != -1 && (int)res.size() > lim + 1) res.resize(lim + 1);
            }
            a = mult(a, a);
            if (lim != -1 && (int)a.size() > lim + 1) a.resize(lim + 1);
            k >>= 1;
        }
        return res;
    }

    // 功能: 分治 NTT 求 n 个多项式连乘 F_1(x) * F_2(x) * ... * F_n(x)
    // 传参: polys 待连乘的多项式向量集合, lim 最高次数限制 (默认 -1 不截断)
    // 返回: 连乘结果多项式系数向量 (时间复杂度 O(N log^2 N), N = sum deg)
    vector<int> poly_prod(const vector<vector<int>>& polys, int lim = -1) const {
        if (polys.empty()) return {1};

        auto dfs = [&](auto&& self, int l, int r) -> vector<int> {
            if (l == r) {
                auto res = polys[l];
                if (lim != -1 && (int)res.size() > lim + 1) res.resize(lim + 1);
                return res;
            }
            int mid = l + (r - l) / 2;
            auto left = self(self, l, mid);
            auto right = self(self, mid + 1, r);
            auto res = mult(left, right);
            if (lim != -1 && (int)res.size() > lim + 1) res.resize(lim + 1);
            return res;
        };

        return dfs(dfs, 0, (int)polys.size() - 1);
    }
};

/*
使用示范:
    NTT ntt;

    // 1. 多项式乘法: (1 + 2x) * (3 + 4x) = 3 + 10x + 8x^2
    vector<int> a = {1, 2}, b = {3, 4};
    vector<int> c = ntt.mult(a, b); // c 为 {3, 10, 8}

    // 2. 多项式快速幂 (带截断): (1 + x)^3 截断到最高 2 次项
    vector<int> p = ntt.poly_pow({1, 1}, 3, 2); // p 为 {1, 3, 3} (即 1 + 3x + 3x^2)

    // 3. n 个多项式连乘 (分治 NTT): (1 + x) * (1 + 2x) * (1 + 3x) = 1 + 6x + 11x^2 + 6x^3
    vector<vector<int>> polys = {{1, 1}, {1, 2}, {1, 3}};
    vector<int> prod = ntt.poly_prod(polys); // prod 为 {1, 6, 11, 6}
*/
```

3. 分拆数 (五边形数定理 + 根号分治 DP)

- 广义五边形数: g_k = k(3k±1)/2, 符号循环: + + - - + + - -
- 根号分治: ≤ B 完全背包; > B 整体+1 与加新数 (B+1) 滚存 DP

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

/**
 * ==========================================================================
 * 模块 1: 五边形数定理 (Pentagonal Number Theorem) 求分拆数
 * ==========================================================================
 * 适用场景: 一次性预处理前 N 项分拆数 p(0 ... N)
```

```
 * 复杂度: 时间 O(N sqrt(N)), 空间 O(N)
 * 递推公式:
 *   p(n) = sum_{k >= 1} (-1)^{k-1} [ p(n - k(3k-1)/2) + p(n - k(3k+1)/2) ]
 *   符号规律: + + - - + + - - ... (奇正偶负)
 */
struct PartitionPentagonal {
    static constexpr int MOD = 998244353;
    int n = 0;
    vector<int> p; // p[i] 为整数 i 的分拆数

    PartitionPentagonal(int max_n = 0) {
        if (max_n >= 0) build(max_n);
    }

    // 功能: 预处理 0 ~ max_n 的分拆数
    // 传参: max_n 预处理上限
    void build(int max_n) {
        n = max_n;
        p.assign(n + 1, 0);
        p[0] = 1;

        // 预生成广义五边形数: k*(3k-1)/2 与 k*(3k+1)/2
        vector<pair<int, int>> pent;
        for (int k = 1;; ++k) {
            long long g1 = 1LL * k * (3LL * k - 1) / 2;
            long long g2 = 1LL * k * (3LL * k + 1) / 2;
            int sign = (k & 1) ? 1 : -1;
            if (g1 <= n) pent.emplace_back((int)g1, sign);
            if (g2 <= n) pent.emplace_back((int)g2, sign);
            if (g1 > n && g2 > n) break;
        }

        for (int i = 1; i <= n; ++i) {
            long long val = 0;
            for (const auto& [g, sign] : pent) {
                if (g > i) break;
                if (sign == 1) {
                    val += p[i - g];
                } else {
                    val -= p[i - g];
                }
            }
            val %= MOD;
            if (val < 0) val += MOD;
            p[i] = (int)val;
        }
    }

    // 功能: 查询 p(x)
    // 传参: x 目标整数
    // 返回: p(x) % MOD (越界返回 0)
    int query(int x) const {
        if (x < 0 || x > n) return 0;
        return p[x];
    }
};

/**
 * ==========================================================================
 * 模块 2: 根号分治 DP 求分拆数
 * ==========================================================================
 * 适用场景: 单点求 p(n) % MOD, 或作为带部件大小/数量限制的分拆数模型模板
 * 复杂度: 时间 O(N sqrt(N)), 空间 O(N)
 * 思路: 阈值 B = sqrt(n)
 *   1. 小数部分 (<= B): 完全背包
 *   2. 大数部分 (> B): 设状态 g[c][j] 为恰有 c 个 > B 的数和为 j
 *      转移: g[c][j] = g[c][j - c] (整体+1) + g[c - 1][j - (B + 1)] (新增一个 B+1)
 *   3. 合并: sum_{k=0}^n f[k] * h[n - k]
 */
struct PartitionSqrt {
    static constexpr int MOD = 998244353;
```

```
    // 功能: 单点计算整数 n 的分拆数 p(n) % MOD
    // 传参: n 目标正整数
    // 返回: p(n) % MOD
    int calc(int n) const {
        if (n < 0) return 0;
        if (n <= 1) return 1;

        int B = sqrt(n);

        // 1. 小数部分 (<= B) 完全背包
        vector<int> f(n + 1, 0);
        f[0] = 1;
        for (int v = 1; v <= B; ++v) {
            for (int j = v; j <= n; ++j) {
                f[j] += f[j - v];
                if (f[j] >= MOD) f[j] -= MOD;
            }
        }

        // 2. 大数部分 (> B) 滚动 DP: 最多有 n / (B + 1) 个数
        vector<int> prev(n + 1, 0), cur(n + 1, 0);
        prev[0] = 1;
        vector<int> h(n + 1, 0);
        h[0] = 1; // c = 0

        int max_c = n / (B + 1);
        for (int c = 1; c <= max_c; ++c) {
            cur.assign(n + 1, 0);
            for (int j = c * (B + 1); j <= n; ++j) {
                int ways = prev[j - (B + 1)] + cur[j - c];
                if (ways >= MOD) ways -= MOD;
                cur[j] = ways;

                h[j] += ways;
                if (h[j] >= MOD) h[j] -= MOD;
            }
            prev = move(cur);
        }

        // 3. 点积合并
        long long ans = 0;
        for (int k = 0; k <= n; ++k) {
            ans = (ans + 1LL * f[k] * h[n - k]) % MOD;
        }
        return (int)ans;
    }
};

/*
使用示范:
    // 1. 五边形数定理 (求前 N 项, 如求 0 ~ 100000 的所有分拆数)
    PartitionPentagonal pp(100000);
    int p5 = pp.query(5); // p(5) = 7 (5, 4+1, 3+2, 3+1+1, 2+2+1, 2+1+1+1, 1+1+1+1+1)

    // 2. 根号分治 DP (单点计算 p(n) 或套用其大数加一/扩项 DP 思想)
    PartitionSqrt ps;
    int p100 = ps.calc(100); // 190569291 % 998244353
*/
```

4. 斯特林数 (O(N^2) 递推打表 + NTT 单行求解)

- 普通幂转下降幂: x^n = ∑ {n, k} x^{underline k}
- 上升幂转普通幂: x^{overline n} = ∑ [n, k] x^k = ∏_{i=0}^{n-1} (x+i)
- 卷积容斥: {n, k} = ∑_{j=0}^k (-1)^{k-j}/(k-j)! * (j^n / j!)

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

/**
 * ==========================================================================
 * 斯特林数 (Stirling Numbers)
```

```
 * ========================================================================
 * 常用恒等式备忘 (赛场速查):
 * 1. 普通幂转下降幂: x^n = sum_{k=0}^n {n, k} * x^{underline k}
 * 2. 上升幂转普通幂: x^{overline n} = sum_{k=0}^n [n, k] * x^k
 * 3. 下降幂转普通幂: x^{underline n} = sum_{k=0}^n (-1)^{n-k} [n, k] * x^k
 * 4. 斯特林反演:
 *    f(n) = sum_{k=0}^n {n, k} g(k) <=> g(n) = sum_{k=0}^n (-1)^{n-k} [n, k] f(k)
 */

/**
 * 模块 1: O(N^2) 斯特林数打表 (适合 N <= 3000 ~ 5000)
 * 提供无符号第一类 [n, k] 与第二类 {n, k}
 */
struct StirlingTable {
    static constexpr int MOD = 998244353;
    int n = 0;
    vector<vector<int>> s1; // 第一类斯特林数 (轮换排列数)
    vector<vector<int>> s2; // 第二类斯特林数 (非空子集划分数)

    StirlingTable(int max_n = 0) {
        if (max_n >= 0) build(max_n);
    }

    // 功能: 预处理 0 ~ max_n 的第一类与第二类斯特林数
    // 传参: max_n 预处理规模
    void build(int max_n) {
        n = max_n;
        s1.assign(n + 1, vector<int>(n + 1, 0));
        s2.assign(n + 1, vector<int>(n + 1, 0));

        s1[0][0] = s2[0][0] = 1;
        for (int i = 1; i <= n; ++i) {
            for (int j = 1; j <= i; ++j) {
                // s1[i][j] = s1[i-1][j-1] + (i-1) * s1[i-1][j]
                s1[i][j] = (s1[i - 1][j - 1] + 1LL * (i - 1) * s1[i - 1][j]) % MOD;
                // s2[i][j] = s2[i-1][j-1] + j * s2[i-1][j]
                s2[i][j] = (s2[i - 1][j - 1] + 1LL * j * s2[i - 1][j]) % MOD;
            }
        }
    }

    // 功能: 查询第一类斯特林数 [x, y]
    int query_s1(int x, int y) const {
        if (x < 0 || y < 0 || y > x || x > n) return 0;
        return s1[x][y];
    }

    // 功能: 查询第二类斯特林数 {x, y}
    int query_s2(int x, int y) const {
        if (x < 0 || y < 0 || y > x || x > n) return 0;
        return s2[x][y];
    }
};

/**
 * 模块 2: O(N log N) / O(N log^2 N) 快速单行求解 (适合 N <= 10^5)
 * 依赖 NTT 卷积与分治
 */
struct StirlingRow {
    static constexpr int MOD = 998244353;

    // 功能: NTT 卷积求解第二类斯特林数第 n 行 {n, 0 ... n}
    // 传参: n 行号, ntt NTT 实例
    // 返回: 系数向量 res, res[k] 为 {n, k} % MOD (复杂度 O(N log N))
    template <typename NTTOp>
    vector<int> row_s2(int n, const NTTOp& ntt) const {
        if (n == 0) return {1};

        vector<int> fac(n + 1, 1), ifac(n + 1, 1);
        for (int i = 1; i <= n; ++i) fac[i] = 1LL * fac[i - 1] * i % MOD;
        ifac[n] = ntt.inv(fac[n]);
```

```
        for (int i = n; i >= 1; --i) ifac[i - 1] = 1LL * ifac[i] * i % MOD;

        vector<int> a(n + 1), b(n + 1);
        for (int i = 0; i <= n; ++i) {
            a[i] = (i & 1) ? (MOD - ifac[i]) % MOD : ifac[i];
            b[i] = 1LL * ntt.ksm(i, n) * ifac[i] % MOD;
        }

        auto c = ntt.mult(a, b);
        c.resize(n + 1);
        return c;
    }

    // 功能: 分治 NTT 求解第一类斯特林数第 n 行 [n, 0 ... n]
    // 传参: n 行号, ntt NTT 实例
    // 返回: 系数向量 res, res[k] 为 [n, k] % MOD (复杂度 O(N log^2 N))
    template <typename NTTOp>
    vector<int> row_s1(int n, const NTTOp& ntt) const {
        if (n == 0) return {1};
        vector<vector<int>> polys(n);
        for (int i = 0; i < n; ++i) {
            polys[i] = {i, 1}; // 生成多项式 (x + i)
        }
        return ntt.poly_prod(polys);
    }
};

/*
使用示范:
    // 1. 小数据打表 O(N^2):
    StirlingTable st(2000);
    int val_s1 = st.query_s1(4, 2); // [4, 2] = 11
    int val_s2 = st.query_s2(4, 2); // {4, 2] = 7

    // 2. 大数据单行求解 O(N log N):
    NTT ntt;
    StirlingRow sr;
    vector<int> r_s2 = sr.row_s2(100000, ntt); // {100000, 0 ... 100000}
    vector<int> r_s1 = sr.row_s1(100000, ntt); // [100000, 0 ... 100000]
*/
```

5. 珂朵莉树 (ODT - Old Driver Tree / Chtholly Tree)

- 核心避坑准则: 双端点切分必须【先 split(r + 1), 再 split(l)】! 否则左端点迭代器可能失效导致 RE!
- 结构 set<array<int, 3>>, 在频繁随机区间推平 assign 下块数期望 O(log log n)

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

/**
 * ============================================================================
 * 珂朵莉树 (ODT - Old Driver Tree / Chtholly Tree)
 * ============================================================================
 * 存储结构: set<array<int, 3>>, 每个节点为 {l, r, v}。
 * 适用场景: 含随机区间赋值 (Assign/推平) 操作的序列维护问题。
 * 核心避坑准则 (赛场必记):
 *   所有区间操作涉及双端点 split 时, 必须【先 split(r + 1), 再 split(l)】!
 *   否则先 split(l) 得到的迭代器可能在 split(r + 1) 执行 erase 时失效, 导致 RE!
 */
struct ODT {
    set<array<int, 3>> s;

    ODT(int n, ll v) {
        s.insert({1, n, (int)v});
    }

    // 功能: 切分包含位置 x 的区间为 [l, x-1] 和 [x, r]
```

```
// 传参: x 切分点 (将成为右区间的左端点)
// 返回: 指向 [x, r] 区间的迭代器
auto split(int x) {
    auto it = prev(s.lower_bound({x + 1, 0, 0}));
    if ((*it)[0] == x) return it;
    auto [l, r, v] = *it;
    s.erase(it);
    s.insert({l, x - 1, v});
    return s.insert({x, r, v}).first;
}

// 功能: 区间推平赋值 [l, r] = v (ODT 维持复杂度的核心)
// 传参: l, r 区间左右界, v 新赋权值
void assign(int l, int r, ll v) {
    auto itr = split(r + 1);
    auto itl = split(l);
    s.erase(itl, itr);
    s.insert({l, r, (int)v});
}

// 功能: 区间加 [l, r] += v
// 传参: l, r 区间左右界, v 增加的权值
void add(int l, int r, ll v) {
    auto itr = split(r + 1);
    auto itl = split(l);
    vector<array<int, 3>> tmp;
    for (auto it = itl; it != itr; ++it) {
        tmp.push_back({(*it)[0], (*it)[1], (int)((*it)[2] + v)});
    }
    s.erase(itl, itr);
    for (auto& node : tmp) {
        s.insert(node);
    }
}

// 功能: 查询区间 [l, r] 第 k 小值 (1-indexed)
// 传参: l, r 区间, k 名次 (1 <= k <= r - l + 1)
// 返回: 第 k 小的值
ll kth(int l, int r, int k) {
    auto itr = split(r + 1);
    auto itl = split(l);
    vector<pair<int, int>> a;
    for (auto it = itl; it != itr; ++it) {
        a.emplace_back((*it)[2], (*it)[1] - (*it)[0] + 1);
    }
    sort(a.begin(), a.end());
    for (const auto& [val, len] : a) {
        k -= len;
        if (k <= 0) return val;
    }
    return -1;
}

// 功能: 查询区间 [l, r] 元素 p 次方和 sum_{i=l}^r (a[i]^p) % mod
// 传参: l, r 区间, p 幂次, mod 模数
// 返回: 区间 p 次方和取模结果
ll sum_pow(int l, int r, int p, ll mod) {
    auto itr = split(r + 1);
    auto itl = split(l);
    auto ksm = [&](ll base, int exp) -> ll {
        ll res = 1 % mod;
        base %= mod;
        while (exp > 0) {
            if (exp & 1) res = (__int128_t)res * base % mod;
            base = (__int128_t)base * base % mod;
            exp >>= 1;
        }
        return res;
    };
    ll ans = 0;
    for (auto it = itl; it != itr; ++it) {
        ll len = (*it)[1] - (*it)[0] + 1;
        ans = (ans + len * ksm((*it)[2], p)) % mod;
    }
```

```
        return ans;
    }
};

/*
使用示范:
    // 1. 初始化区间 [1, 100000] 全为 0
    ODT odt(100000, 0);

    // 2. 区间加: [1, 10] += 5
    odt.add(1, 10, 5);

    // 3. 区间推平 (核心): [3, 8] = 7
    odt.assign(3, 8, 7);

    // 4. 查询第 k 小: [1, 10] 第 4 小
    ll val = odt.kth(1, 10, 4);

    // 5. 查询区间平方和对 1e9+7 取模
    ll sum2 = odt.sum_pow(1, 10, 2, 1000000007LL);
*/
```

6. 主席树 (可持久化权值线段树 / 静态区间第 k 小 / 树上第 k 小)

- 基础: roots[r] 与 roots[l-1] 差分二分 | 单次插/查 O(log M), 空间 O(N log M) 动态开点
- 扩展: 区间 ≤ x 计数 count_le; 树上路径差分 u + v - lca - fa_lca

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

/**
 * ============================================================================
 * 主席树 (可持久化权值线段树 - Persistent Segment Tree)
 * ============================================================================
 * 适用场景: 静态区间第 k 小 / 区间值域统计 / 树上路径第 k 小 (结合 LCA 差分)
 * 复杂度:
 *   - 空间复杂度: O(N log M) 动态开点 (M 为离散化后值域大小)
 *   - 时间复杂度: 单次单点插入 O(log M), 单次区间查询 O(log M), 整树构建 O(N log N)
 * 特性: 内置离散化与前缀版本链, 纯血 vector 动态开点, 多测自闭环绝对安全。
 */
struct PersistentSegTree {
    struct Node {
        int lc = 0, rc = 0, cnt = 0;
    };

    int n = 0, m = 0;
    vector<int> roots;
    vector<Node> tr;
    vector<int> vals; // 离散化去重原值表

    PersistentSegTree() {}
    PersistentSegTree(const vector<int>& a) {
        build(a);
    }

    // 功能: 在历史版本 p 基础上, 在值域位置 pos 增加 1 生成新节点
    // 传参: p 历史版本节点编号, l, r 当前值域区间, pos 待插入值域下标 (1 <= pos <= m)
    // 返回: 新开节点编号
    int insert(int p, int l, int r, int pos) {
        int u = tr.size();
        tr.push_back(tr[p]);
        tr[u].cnt++;
        if (l == r) return u;
        int mid = l + (r - l) / 2;
        if (pos <= mid) {
            tr[u].lc = insert(tr[p].lc, l, mid, pos);
        } else {
            tr[u].rc = insert(tr[p].rc, mid + 1, r, pos);
```

```
    }
    return u;
}

// 功能: 在两版本 u (右) 与 v (左) 间做差, 二分查找第 k 小位置 (1-indexed)
// 传参: u 右端点根编号, v 左端点前驱根编号, l, r 当前值域, k 名次 (1 <= k <= cnt)
// 返回: 离散化值域下标 (1 <= res <= m)
int query(int u, int v, int l, int r, int k) const {
    if (l == r) return l;
    int left_cnt = tr[tr[u].lc].cnt - tr[tr[v].lc].cnt;
    int mid = l + (r - l) / 2;
    if (k <= left_cnt) {
        return query(tr[u].lc, tr[v].lc, l, mid, k);
    } else {
        return query(tr[u].rc, tr[v].rc, mid + 1, r, k - left_cnt);
    }
}

// 功能: 用原数组一键构建前缀主席树 (下标 1 ~ a.size()-1)
// 传参: a 原始数组向量 (1-indexed)
void build(const vector<int>& a) {
    n = a.size() - 1;
    roots.assign(n + 1, 0);
    tr.clear();
    tr.emplace_back(); // 0 号空节点 (lc=0, rc=0, cnt=0)

    // 离散化
    vals.assign(a.begin() + 1, a.end());
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    m = vals.size();

    for (int i = 1; i <= n; ++i) {
        roots[i] = insert(roots[i - 1], get_pos(a[i]));
    }
}

// --- 一行小封装入口 ---
int insert(int p, int pos) { return insert(p, 1, m, pos); }
int get_pos(int x) const { return lower_bound(vals.begin(), vals.end(), x) - vals.begin() + 1; }
int query_kth(int l, int r, int k) const { return vals[query(roots[r], roots[l - 1], 1, m, k) - 1]; }

// ============================================================================
// 可选扩展 1: 区间值域计数 / 二维数点 (Count / Rank)
// ============================================================================
// 功能: 查询区间 [l, r] 内值域在 [ql, qr] 范围的元素个数
int query_count(int u, int v, int l, int r, int ql, int qr) const {
    if (ql > r || qr < l || ql > qr) return 0;
    if (ql <= l && r <= qr) return tr[u].cnt - tr[v].cnt;
    int mid = l + (r - l) / 2;
    return query_count(tr[u].lc, tr[v].lc, l, mid, ql, qr) +
           query_count(tr[u].rc, tr[v].rc, mid + 1, r, ql, qr);
}
// 一行小封装: 区间 <= x 的元素个数
int count_le(int l, int r, int x) const {
    int qr = upper_bound(vals.begin(), vals.end(), x) - vals.begin();
    return query_count(roots[r], roots[l - 1], 1, m, 1, qr);
}
// 一行小封装: 区间值在 [x, y] 范围内的元素个数
int count_range(int l, int r, int x, int y) const { return count_le(l, r, y) - count_le(l, r, x - 1); }

// ============================================================================
// 可选扩展 2: 树上路径第 k 小 (Tree Path Kth)
// ============================================================================
// 功能: 树上前缀差分 (u + v - lca - fa_lca) 二分第 k 小
int query_tree(int u, int v, int lca, int fa_lca, int l, int r, int k) const {
    if (l == r) return l;
    int left_cnt = tr[tr[u].lc].cnt + tr[tr[v].lc].cnt - tr[tr[lca].lc].cnt - tr[tr[fa_lca].lc].cnt;
    int mid = l + (r - l) / 2;
    if (k <= left_cnt) return query_tree(tr[u].lc, tr[v].lc, tr[lca].lc, tr[fa_lca].lc, l, mid, k);
    else return query_tree(tr[u].rc, tr[v].rc, tr[lca].rc, tr[fa_lca].rc, mid + 1, r, k - left_cnt);
}
// 一行小封装: 树上路径第 k 小原值 (建树时 roots[u] = insert(roots[fa], get_pos(val[u])))
```

```
    int query_path_kth(int u, int v, int lca, int fa_lca, int k) const {
        return vals[query_tree(roots[u], roots[v], roots[lca], roots[fa_lca], 1, m, k) - 1];
    }
};

/*
使用示范:
    // a 必须是 1-indexed (a[0] 占位)
    vector<int> a = {0, 3, -1, 5, 2, 4, 2, 5, 10, -5};

    // 1. 初始化并构建前缀主席树
    PersistentSegTree pst(a);

    // 2. 一行小封装: 查询区间 [2, 6] 第 3 小的值
    int val = pst.query_kth(2, 6, 3);

    // 3. 可选扩展 1: 查询区间 [2, 6] 内 <= 4 的元素个数
    int cnt = pst.count_le(2, 6, 4);

    // 4. 可选扩展 2: 树上路径第 k 小 (需配合树剖/倍增求 LCA)
    // int ans = pst.query_path_kth(u, v, lca, fa[lca], k);
*/
```